



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Отчёт о лабораторной работе № 6 **по курсу «Численные методы»**

**"Метод наискорейшего спуска поиска минимума функции многих
переменных"**

Студент: Ивенкова О. В.

Группа: ИУ9-62Б

Преподаватель: Домрачева А. Б.

Москва, 2026

1 Постановка задачи

Дано:

Функция $f(x_1, \dots, x_n)$, начальная точка x^0 .

Задание:

1. Найти минимум функции двух переменных с точностью $\varepsilon = 0,001$, начиная итерации из точки x^0 .
2. Найти минимум аналитичности.
3. Сравнить полученные результаты.

Индивидуальный вариант №9:

$$x_1^4 + x_2^2 + 2x_2 + \cos(0.2x_1 + 0.15x_2), \quad x^0 = (0, -1)$$

2 Теоретические сведения

Метод наискорейшего спуска является итерационным. Пусть для функции $f(x_1, x_2, \dots, x_n)$ (или в векторной форме $f(x)$) на k -м шаге имеем приближение к минимуму $x^k = (x_1^k, \dots, x_n^k)$.

Рассмотрим функцию одной переменной

$$\varphi_k(t) = f(x_1^k - t \frac{\partial f}{\partial x_1}(x^k), \dots, x_n^k - t \frac{\partial f}{\partial x_n}(x^k)) = f(x^k - t \operatorname{grad} f(x^k))$$

где вектор $\operatorname{grad} f(x^k) = (\frac{\partial f}{\partial x_1}(x^k), \dots, \frac{\partial f}{\partial x_n}(x^k))$ - градиент функции f в точке x^k . Функция $\varphi_k(t)$ представляет собой ограничение исходной функции $f(x)$ на прямую градиентного (наискорейшего) спуска, проходящую через точку k -го приближения x^k (рис. 1).

Минимум функции $\varphi_k(t)$ можно найти любым методом одномерной оптимизации. Обозначим эту точку минимума через t^* . Теперь для следующего приближения к точке экстремума полагаем

$$x^{k+1} = x^k - t^* \operatorname{grad} f(x^k) = (x_1^k - t^* \frac{\partial f}{\partial x_1}(x^k), \dots, x_n^k - t^* \frac{\partial f}{\partial x_n}(x^k))$$

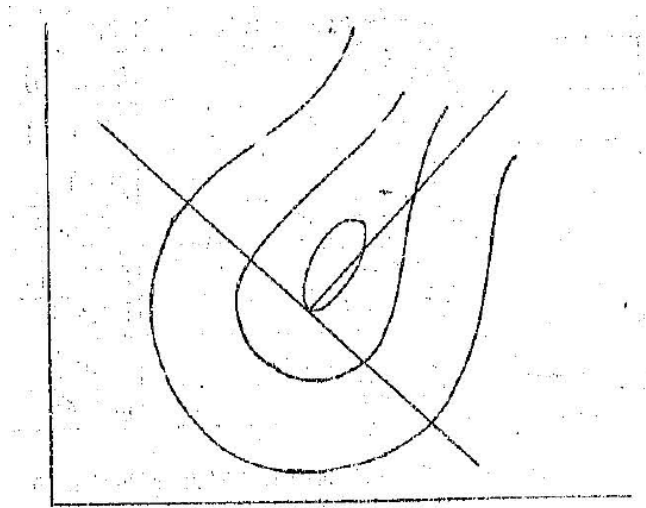


Рис. 1

Процесс поиска минимума продолжаем до тех пор, пока $\|gradf(x^k)\| = \max_{1 \leq i \leq n} \left| \frac{\partial f}{\partial x_i}(x^k) \right|$ не станет меньше допустимой погрешности ε .

В большинстве случаев точно искать минимум функции $\varphi_k(t)$ не нужно и достаточно ограничиться одним приближением к x_n , построенным, например, по методу парабол. Тогда особенно простым вид итерации будет в двумерном случае:

$$(x_{k+1}, y_{k+1}) = \left(x_k - t^* \frac{\partial f}{\partial x}, y_k - t^* \frac{\partial f}{\partial y} \right)$$

где $t^* = -\frac{\varphi'_k(0)}{\varphi''_k(0)}$, $\varphi'_k(0) = -\left(\frac{\partial f}{\partial x}\right)^2 - \left(\frac{\partial f}{\partial y}\right)^2$, $\varphi''_k(0) = \frac{\partial^2 f}{\partial x^2} \left(\frac{\partial f}{\partial x}\right)^2 + 2 \frac{\partial^2 f}{\partial x \partial y} \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} + \frac{\partial^2 f}{\partial y^2} \left(\frac{\partial f}{\partial y}\right)^2$ (все производные берутся в точке (x_k, y_k)).

3 Практическая часть

Исходный код программы представлен в листинге 1.

Листинг 1: Код программы

```
1 import math
2
3 def f(x1, x2):
4     return x1**4 + x2**2 + 2*x2 + math.cos(0.2*x1 + 0.15*x2)
5
6 def df_dx1(x1, x2):
7     return 4*x1**3 - 0.2*math.sin(0.2*x1 + 0.15*x2)
8
9 def df_dx2(x1, x2):
10    return 2*x2 + 2 - 0.15*math.sin(0.2*x1 + 0.15*x2)
11
12 def d2f_dx1(x1, x2):
13    return 12*x1**2 - 0.04*math.cos(0.2*x1 + 0.15*x2)
14
15 def d2f_dx2(x1, x2):
16    return 2 - 0.0225*math.cos(0.2*x1 + 0.15*x2)
17
18 def d2f_dx1dx2(x1, x2):
19    return -0.03*math.cos(0.2*x1 + 0.15*x2)
20
21 def steepest_descent(x1, x2, eps=0.001, max_iter=1000):
22    print("\n{:>3}  {:>12}  {:>12}  {:>12}".format("k", "x1", "x2", "f(x)"))
23
24    for k in range(max_iter):
25        g1 = df_dx1(x1, x2)
26        g2 = df_dx2(x1, x2)
27
28        print("{:>3}  {:>12.6f}  {:>12.6f}  {:>12.6f}".format(k, x1, x2, f(
29            x1, x2)))
30
31        if max(abs(g1), abs(g2)) < eps:
32            break
33
34        phi1 = -(g1**2 + g2**2)
35
36        phi2 = (
37            d2f_dx1(x1, x2) * g1**2 +
38            2 * d2f_dx1dx2(x1, x2) * g1 * g2 +
39            d2f_dx2(x1, x2) * g2**2
```

```

40
41         t = -phi1 / phi2
42
43         x1 = x1 - t * g1
44         x2 = x2 - t * g2
45
46     return x1, x2, f(x1, x2)
47
48 x1_0 = 0
49 x2_0 = -1
50
51 x1_chisl, x2_chisl, f_chisl = steepest_descent(x1_0, x2_0)
52
53
54 #-----
55 from sympy import symbols, sin, cos, nsolve
56
57 x1, x2 = symbols('x1 x2')
58
59 eq1 = 4*x1**3 - 0.2*sin(0.2*x1 + 0.15*x2)
60 eq2 = 2*x2 + 2 - 0.15*sin(0.2*x1 + 0.15*x2)
61
62 sol = nsolve((eq1, eq2), (x1, x2), (0, -1))
63
64 f = x1**4 + x2**2 + 2*x2 + cos(0.2*x1 + 0.15*x2)
65
66 val = f.subs({x1: sol[0], x2: sol[1]})
67 f_analitic = val.evalf()
68
69 #-----
70
71 print(f"\nanalitic \t \t \t \t | \tnumerical")
72 print(f"x1_a\t\tx2_a\t\tf_a(x1, x2)\t | \tx1\t\tx2\t\tf(x1, x2)")
73 print(f"{sol[0]:.6f}\t{sol[1]:.6f}\t{f_analitic:.8f}\t|A\t{x1_chisl:.6f}
74         \t{x2_chisl:.6f}\t{f_chisl:.8f}")
75
76 print(f"\nDifference x:\n", abs(sol[0] - x1_chisl), "\t", abs(sol[1] -
77         x2_chisl))
78 print(f"\nDifference result min:\n {abs(f_chisl - f_analitic):.12f}\n")

```

Вывод программы после запуска:

```
k      x1      x2      f(x)
0      0.000000 -1.000000 -0.011229
1      -0.045408 -1.034056 -0.012285
2      -0.070276 -1.000509 -0.013415
3      -0.115160 -1.034205 -0.014483
4      -0.136220 -1.004178 -0.015416
5      -0.169161 -1.027876 -0.016027
6      -0.181114 -1.009491 -0.016388
7      -0.197546 -1.020409 -0.016545
8      -0.202256 -1.012743 -0.016609
9      -0.208305 -1.016498 -0.016630
10     -0.209815 -1.013976 -0.016637
11     -0.211706 -1.015112 -0.016639
12     -0.212153 -1.014360 -0.016640

analytic      |      numerical
x1_a          x2_a      f_a(x1, x2) |      x1      x2      f(x1, x2)
-0.213109     -1.014518 -0.01664016 |      -0.212153 -1.014360 -0.01663991

Difference x:
0.000956882652159846      0.000157456278627288

Difference result min:
0.000000250874
```

4 Выводы

В процессе выполнения лабораторной работы был изучен и реализован метод наискорейшего спуска поиска минимума функции многих переменных. По результатам сравнения полученных значений аналитичности и численно можно сделать вывод о том, что они подтверждают правильность выполненных вычислений и эффективность численных методов оптимизации для решения нелинейных задач.